



Stencil Project: Heat equation using MPI + OpenMP

LANGAGES DU PARALLÉLISME

25-26

Author: Santiago Martín Serrano

Professeur: Alexandre Denis

Contents

1. Introduction	3
2. MPI Version	4
2.1. Initialization	4
2.2. Halo Exchange and Stencil Step	5
3. OpenMP version	6
4. MPI + OpenMP Version	7
5. Performance Results	8
5.1. MPI Version Performance	8

1. Introduction

This work presents an implementation for the Stencil Problem (2D).

This problem consists on a matrix that is iteratively updated as the heat values in the edges are propagated. At first (iteration=0), we initialize the edges of the matrix with values from 0 to the matrix height (or matrix width, both values are the same, we treat a square matrix). The values inside the matrix are set to 0.

On each step, the stencil operation for 2D is performed, and each point is updated using the values of the neighbours of the previous iteration (the neighbour size can depend, here we will use 5 format stencil, up, down, left, right).

This operation is repeated until the difference between the previous values and the current iteration values, for each point, is less than a given value (convergence).

In this work we dive into three types of implementations:

1. **A distributed memory implementation (OpenMPI).** Each process has its own set of information, different for each process. The OpenMPI library offers efficient communication between processes using a wide range of functions. For the stencil problem, the matrix is divided into rows, and each process receives a subset of rows. This approach shows good performance due to data locality, as the row-size calculation is small enough to fit in L2 or L3 cache.
2. **A shared memory implementation (OpenMP).** This version uses a single process, and multiple threads sharing the same memory space. Unlike the MPI implementation, OpenMP distributes *iterations of the stencil computation* among threads over the whole matrix.
3. **A hybrid implementation using MPI and OpenMP.** Data is distributed between MPI processes, and within each process the stencil loops are parallelized using OpenMP threads. This approach introduces additional constraints and potential issues that will be discussed later.

2. MPI Version

As mentioned before, when using MPI each process has its own context. We need communication techniques between the processes, so each process receives its rows, and at the end the result of each process is gathered.

We can divide the MPI implementation code in 3 sections. The program execution flow is as follows:

1. `MPI_Init`.
2. `stencil_init_mpi()`, where only the rank 0 process initializes the global matrix and then distributes the rows to the other processes using `MPI_Scatterv`.
3. Main loop, where `stencil_step` is computed at each iteration (function where the halos are exchanged) and then the convergence is checked every 100 iterations.

2.1. Initialization

The `stencil_init_mpi` function is called at the beginning of the program. As said before, in MPI each process owns only its local rows and has no direct information about the data stored in other processes. The rank 0 process initializes the global matrix, and then distributes the data using `MPI_Scatterv`. This *blocking* function relies on two arrays: `sendcounts`, which stores the number of elements assigned to each process, and `displs` stores the physical address that each process will take as the 0 position of its vector (the offset).

`MPI_Scatterv` is used in the initialization because it allows a variable number of elements to be sent to each process. This is necessary because the number of processes *module* the number of rows might be $\neq 0$, meaning that some processes must handle an extra row. This approach is both flexible and simple to implement.

Also it's worth noting that `size_x` are the number of columns, and `size_y` the number of rows. `size_x` remains the same like in the sequential version of stencil, also being a global variable. However `size_y` is the variable that has been updated to `->local_ny` (local rows for each process, not global). In the stencil operation there is a double

for loop in order to go through the matrix. So one of the for loops has been adapted to iterate until `<local_ny`. This initialization part was not particularly high in difficulty.

2.2. Halo Exchange and Stencil Step

The matrix has been distributed into rows, that each process will compute. To update the value of each point of the row, we need the values of the neighbour points. But because each process only know its rows, when a process wants to update the value of its first row, the last row that the previous rank has, is needed. This is called the superior halo, each process must have it. Also the inferior halo.

Up to this point, each process has a set of rows, and two empty rows (the first one, and the last one). This is because now for each iteration (in the `stencil_step` function), those rows will be filled with the halos (the rows from other processes).

If we have a given process, for example let's say the rank 1 process, its superior halo (empty at the moment) will be filled with the last *real* row of the previous process (in this case, rank 0). And the inferior halo (empty at the moment) of the rank 1 will be filled with the first real row of the rank 2 process. This is done for all the processes. Rank 0 and rank `ny_local` are special cases. For rank 0, the superior halo will always remain empty, and for the rank `ny_local` its inferior halo will also remain empty.

For the rank 0 process, as an example:

- `values[0]` ->superior halo
- `values[1 * size_x ... local_ny * size_x]` ->real rows
- `values[local_ny + 1]` ->inferior halo

The halo exchange is made thanks to the `MPI_Sendrecv` function. Two calls of this function are needed. In the first call, each process sends its real first row, to its previous process. And also in this first call, each process receives its inferior halo, from other process that has sent first it's first row.

Then the second `MPI_Sendrecv` call does the same thing but inversely. Each process sends its last real row, and also each one receives its superior halo.

MPI_SendI and MPI_RecvI could have been used, but this implementation is simpler and we make sure no deadlocks or blocking issues are produced.

Every 100 stencil_step iterations, in the main, an if condition is checked to see if the convergence has already been reached. Once the loop has made 100 iterations, MPI_Allreduce is called to make a logical AND between all the convergence_local variables for each process. If some process rows have not converged yet, the condition will not be true, and more iterations will be made. We use MPI_Allreduce and not MPI_Reduce because we want all the processes to have the result, not only the rank 0. This way every process knows if it has to exit the for loop in the main or not.

3. OpenMP version

In this version, OpenMP directives are implemented specifically in the for loop of the stencil_step function.

The problem in OpenMP implementation is not the double for loop parallelization, but the convergence check. We use the following directive:

```
pragma omp parallel for collapse(2) reduction(:convergence)
```

We use "parallel for" to create a set of threads that distribute the work of the matrix. Because each calculation of the values of the matrix only depends on the previous state (prev_values), all iterations in the loop are independent. This allows us to run them at the same time on different CPU cores.

Also, we use collapse(2) because our stencil has two embedded loops: one for the rows (y) and one for the columns (x). By using collapse(2), we tell OpenMP to merge these two loops into a single large iteration space. The benefit is that it gives OpenMP more "work units" to distribute.

The reduction(:convergence) part is the most critical part for the algorithm to work correctly. Because if we didn't use reduction, all the threads would try to write to the same memory address at the same time (this implies a race condition), which would lead to incorrect results.

With reduction, OpenMP gives each thread a private copy of the variable. At the end of the loop a logical AND across all private copies is done to update the global convergence variable.

4. MPI + OpenMP Version

For this implementation, we start from the MPI version and add the same directive used in the OpenMP version, but with `private(x)`.

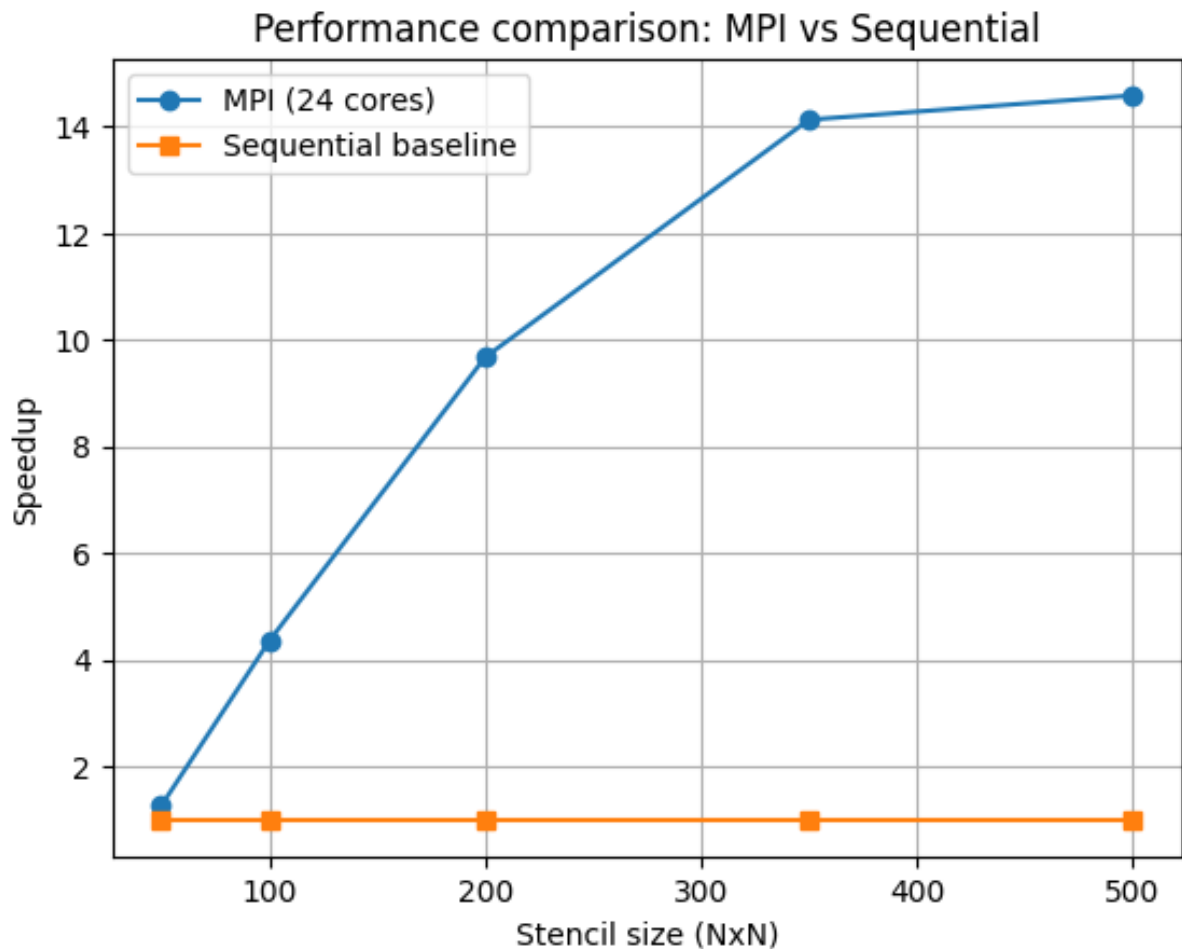
We place that additional part, because if `x` was shared, all threads will try to change the same `x` at the same time. For example, while the first thread is trying to do its work, the second thread might change the value of `x`. This creates a race condition.

By using `private(x)`, we tell OpenMP that every thread needs its own copy of `x`. So each thread can count from 1 to 500 independently without interfering with the other threads.

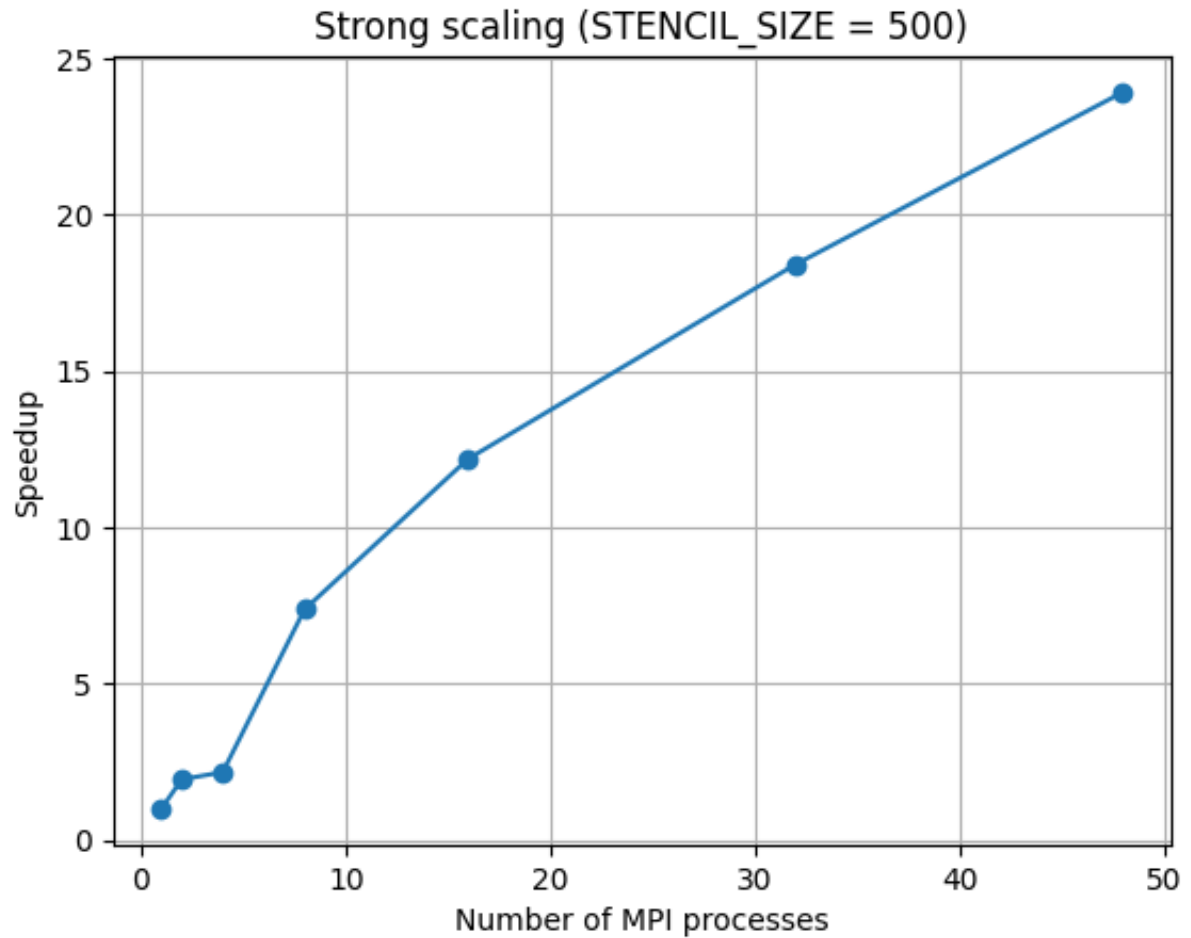
For the outer loop `y`, OpenMP already makes it private because the `#pragma` directive is above.

5. Performance Results

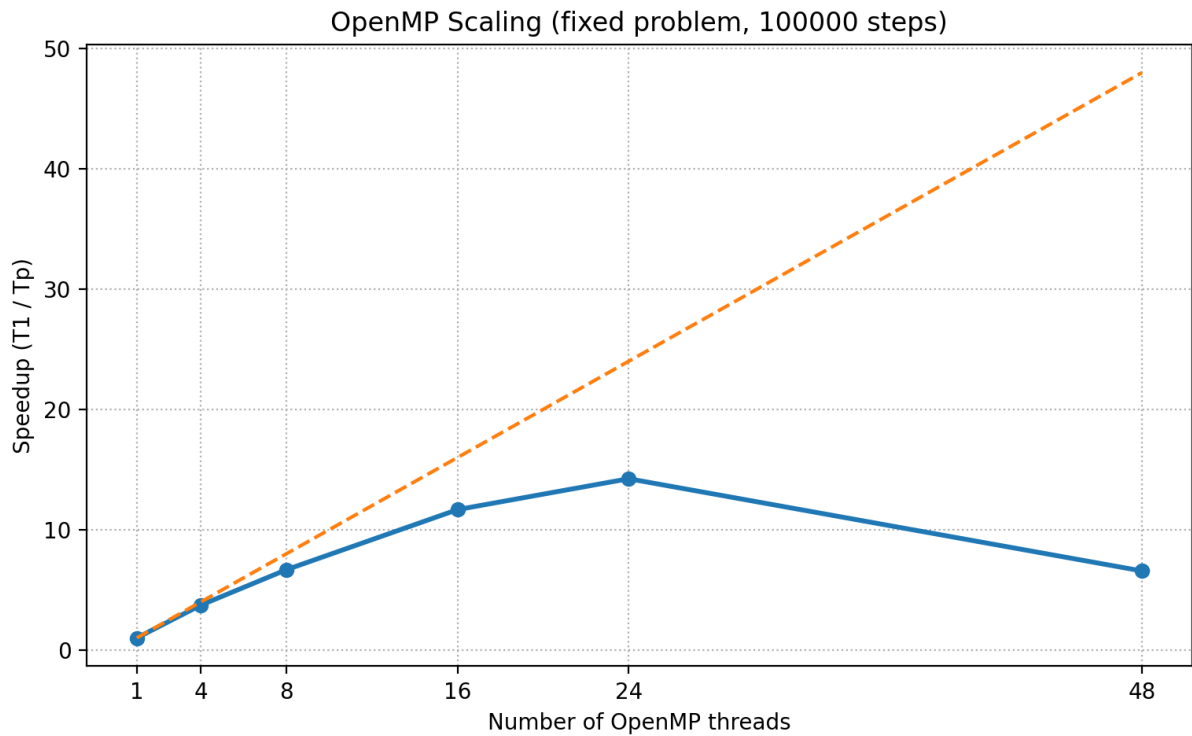
5.1. MPI Version Performance



In this graphic we have in the X axis the stencil size, measured with sizes of 500, 350, 200, 100 and 50. In the Y axis the speedup is presented (sequential time / MPI). This was performed with 24 cores, one cluster node. We observe that the speedup is not that high at first with a 50x50 matrix size, so the sequential version can even catch up. The communication costs of MPI's version weigh. However, as the matrix size increases, we observe a bigger speedup.



For this second graphic, the number of processes / cores available have been measured (because in the pure MPI version, a process corresponds to a core) vs the speedup. As the number of processes/cores increases, the performance is better. This is all for a 500 stencil size. The performance with more cores will probably slow down due to excessive communication times, and the saturation of the memory bandwidth.



Here we measure the number of threads in the X axis, and the speedup in the Y axis. We use 1 node, 24 cores. We see that decrease at the end, because since the physical node only has 24 cores, when requesting 48 threads the operating system has to be swapping threads in each core (context switching)...

Also, it is important to remark that for the OpenMP + MPI version, multiple combinations of threads and MPI processes values has been tested. 2 threads per process and 24 processes doesn't work well, because we find each core saturated with switching context (0.81 Gflops/s). The sweet spot we found is 6 threads and 4 processes, for 2 nodes reserved in the cluster.

For 6 threads and 4 processes

- Number of MPI processes: 4
- steps = 100000
- time = 7.87547e+06 usecs.
- gflops = 19.0465